

Git colaborativo — Informe de completitud

Repositorio: <https://github.com/arielsch74/tp01live> · Alumno: arielsch74 · 5 de agosto de 2026

18 / 18 requisitos verificados contra el estado real del repositorio

1. Qué es este documento y cómo se produjo

Este informe cruza **cada requisito del enunciado del TP01** contra **lo que el repositorio efectivamente tiene**. La distinción no es retórica: cada fila de la tabla de la sección 2 sale de una consulta a la API de GitHub o a Git ejecutada en el momento de generar este PDF, no de lo que creemos haber hecho. Si un requisito no estuviera cumplido, la fila lo diría.

El chequeo está scripteado y es reproducible: el script vive en `tools/verificar.py`, dentro del propio repositorio, y cualquiera con acceso al repositorio público puede volver a correrlo —las consultas están citadas una por una en la columna *Fuente*— y obtener las mismas respuestas. La única afirmación de este documento que no se puede verificar contra la API es su propia presencia en el repositorio, porque se incorpora por Pull Request después de haber sido generado.

Por qué se verifica por API y no de memoria. La ejecución de este TP fue asistida por IA (declarado en detalle en `decisiones.md`). Un agente puede reportar con total seguridad que hizo algo que no hizo — y ése es exactamente el riesgo que este informe está diseñado para cerrar: la fuente de verdad es el servidor de GitHub, no el relato de quien ejecutó. Durante este trabajo el riesgo se materializó dos veces, con capturas que el proceso dio por buenas y estaban arruinadas; se detectaron abriendo los archivos, no preguntando.

2. Requisito por requisito

#	Requisito del enunciado	Estado	Fuente (consulta real)	Qué devolvió
1.a	Repositorio público en la plataforma elegida	CUMPLE	GET /repos/arielsch74/tp01live	visibility = public, private = False, default_branch = main
1.b	.gitignore en la raíz	CUMPLE	GET /repos/arielsch74/tp01live/contents/.gitignore	presente en la raíz de main, 200 bytes
2.a	main protegida: todo cambio entra por Pull Request	CUMPLE	GET /repos/arielsch74/tp01live/branches/main/protection	required_pull_request_reviews está activo → GitHub rechaza cualquier push directo a main
2.b	Imposible pushear directo — tampoco para el administrador (sin bypass)	CUMPLE	GET /repos/arielsch74/tp01live/branches/main/protection	enforce_admins.enabled = True (equivale a «Do not allow bypassing the above settings»)

2.c	Evidencia del rechazo del push directo	CUMPLE	GET /repos/arielsch74/tp01live/contents/img/01-push-rechazado.png	captura de la ventana de terminal real en el instante del rechazo, versionada en el repo
3.a	Al menos 2 Pull Requests mergeados	CUMPLE	GET /repos/arielsch74/tp01live/pulls?state=all	4 PRs mergeados
3.b	Uno de los PRs requirió resolver un conflicto de merge	CUMPLE	GET /repos/arielsch74/tp01live/pulls/<n>/commits (commit con 2 padres)	PR #3 «Título del README: "Proyecto IngSoft3 - versión B"» contiene un commit de merge con 2 padres (1edc426): es la resolución del conflicto
3.b.2	Las dos ramas del conflicto nacieron del mismo commit (conflicto real)	CUMPLE	GET /repos/arielsch74/tp01live/compare/<padre1>...<padre2> → merge_base_commit	los dos padres del commit de resolución tienen como ancestro común a 301f094: las dos ramas salieron del mismo main. Si la rama B hubiera nacido de la A no habría habido conflicto y el ejercicio no probaría nada
3.c	Todo cambio entra por PR (incluidos evidencias.md y decisiones.md)	CUMPLE	GET /repos/arielsch74/tp01live/commits	4 de 6 commits de main son merges de PR (squash). Los que no lo son son anteriores a la protección: el commit inicial del repositorio y el del .gitignore (§4.3 de la guía va antes de §4.4)
3.d	Los dos archivos de la entrega llegaron a main por Pull Request	CUMPLE	GET /repos/<r>/commits?path=decisiones.md	decisiones.md entró en el commit «docs: agrega evidencias.md, decisiones.md y las capturas del TP1 (#4)», que es un merge de PR
3.e	Ramas de feature borradas después de integrar	CUMPLE	GET /repos/arielsch74/tp01live/branches	ramas vivas: main
4.a	Tag v1.0.0 (semver) sobre main	CUMPLE	GET /repos/arielsch74/tp01live/git/ref/tags/v1.0.0 + git merge-base --is-ancestor	tag anotado apuntando al commit d1baff1, que es ancestro de main
4.b	Release publicada con notas de qué incluye	CUMPLE	GET /repos/arielsch74/tp01live/releases/tags/v1.0.0	release «v1.0.0» publicada (draft = False, prerelease = False), notas de 1686 caracteres
5.b	«decisiones.md» en la raíz del repositorio	CUMPLE	GET /repos/arielsch74/tp01live/contents/decisiones.md	presente en main, 12966 bytes
5.c	«evidencias.md» en la raíz del repositorio	CUMPLE	GET /repos/arielsch74/tp01live/contents/evidencias.md	presente en main, 4108 bytes

5.d	Las cuatro capturas 📸 están versionadas en el repositorio	CUMPLE	GET /repos/arielsch74/tp01live/contents/img/*	4/4 presentes
5.e	evidencias.md referencia y describe las cuatro capturas	CUMPLE	contenido de evidencias.md en main	4/4 capturas referenciadas desde el markdown
5.f	decisiones.md cubre los tres puntos que pide el enunciado	CUMPLE	contenido de decisiones.md en main	4/4 secciones presentes

3. El estado real del repositorio

3.1 Historial de `main`

```
eba92b6 docs: agrega evidencias.md, decisiones.md y las capturas del TP1 (#4)
d1baff1 docs: renombra el título del README a Proyecto IngSoft3 - versión B (#3)
a8e3a5c docs: renombra el título del README a Proyecto IngSoft3 - versión A (#2)
301f094 Agrega sección de instalación al README (#1)
fcba76e chore: agrega .gitignore base del proyecto
bd6b83e Initial commit
```

Los commits que terminan en `(#n)` son merges de Pull Request con estrategia *squash*: un commit por PR. Los dos que no —el commit inicial y el del `.gitignore`— son **anteriores a la protección**, porque la guía crea el `.gitignore` en §4.3 y protege `main` en §4.4. Desde que la protección existe, no hay ningún commit en `main` que no haya entrado por un PR.

3.2 Pull Requests mergeados

PR	Título	Rama	Mergeado (UTC)	Integración
#1	Agrega sección de instalación al README	feature/seccion-instalacion	2026-08-06T00:59:08Z	limpio
#2	Título del README: "Proyecto IngSoft3 - versión A"	feature/titulo-a	2026-08-06T00:59:52Z	limpio
#3	Título del README: "Proyecto IngSoft3 - versión B"	feature/titulo-b	2026-08-06T01:05:08Z	conflicto resuelto a mano
#4	Agrega evidencias.md, decisiones.md y las capturas del TP1	feature/entrega-tp01	2026-08-06T01:08:51Z	limpio

Ninguna rama de `feature` quedó viva: se borraron al mergear, y hoy el repositorio tiene únicamente `main`. Las ramas de `feature` son descartables, no mascotas.

Nota sobre el conteo: este informe se generó con 4 PRs mergeados. El siguiente —el que incorpora este mismo PDF al repositorio— no puede figurar acá, por la misma razón por la que un informe no puede verificar su propia llegada: se genera antes de existir en `main`.

3.3 Protección de `main`

```
required_pull_request_reviews = activo
required_approving_review_count = 0
enforce_admins.enabled = True
allow_force_pushes = False, allow_deletions = False
```

`enforce_admins` activo es el equivalente de *Do not allow bypassing the above settings*: la regla alcanza al dueño del repositorio. Las aprobaciones obligatorias están en **cero** a propósito: el TP es individual y GitHub no permite aprobar el propio PR (la API responde 422 Can not approve your own pull request), así que exigir una aprobación dejaría todos los PRs imposibles de mergear. En un equipo real ese número va en 1 o más.

3.4 El conflicto, verificado

```
41a496a (1 padres) docs: renombra el título del README a Proyecto IngSoft3 - versión B · 1edc426 (2
padres) fix: resuelve el conflicto del título tomando la versión B

padres = 41a496a, a8e3a5c · merge_base = 301f094
```

El commit `1edc426` tiene **dos padres**: es el merge que resolvió el conflicto, y es la prueba estructural —no anecdótica— de que ese PR pasó por una resolución. El ancestro común de sus dos padres es `301f094`: el `main` del que salieron las dos ramas. Ese dato es el que descarta el error clásico del ejercicio —que la rama B nazca de la A—, en cuyo caso no habría habido conflicto alguno.

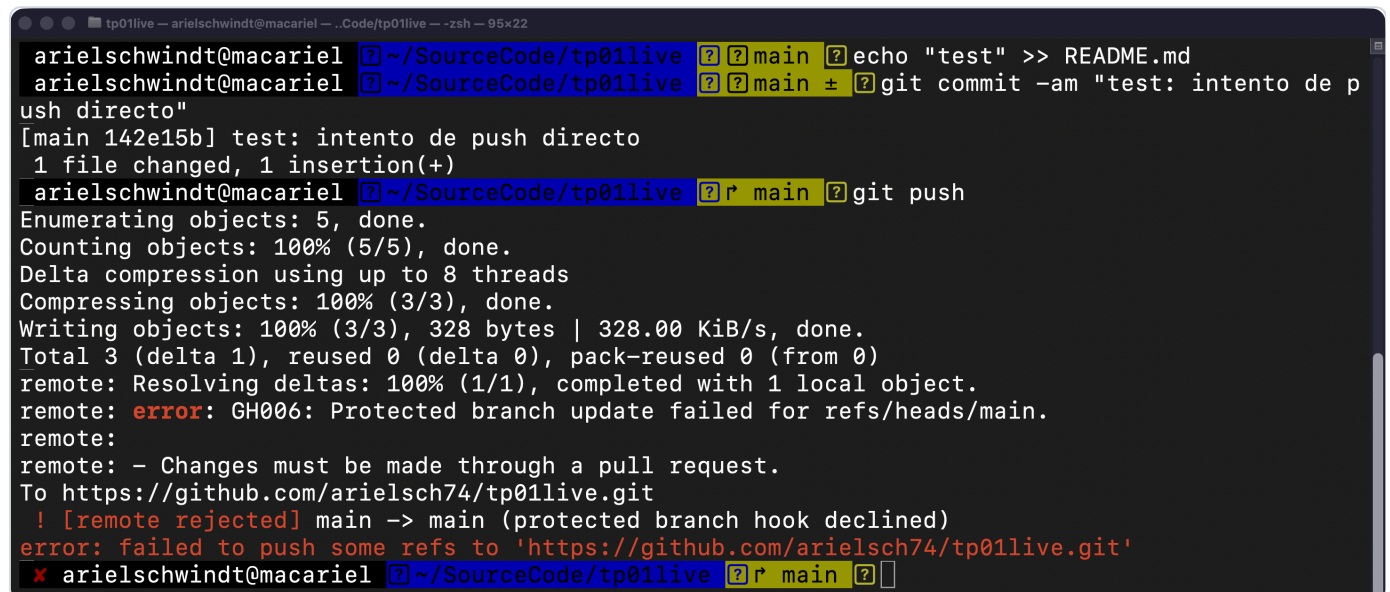
3.5 Tag y release

```
tag          v1.0.0 (anotado) -> commit d1baff1 [ancestro de main: sí]
mensaje      Primera versión estable del TP1: flujo de trabajo del equipo funcionando
release      release «v1.0.0» publicada (draft = False, prerelease = False), notas de 1686 caracteres
url          https://github.com/arielsch74/tp01live/releases/tag/v1.0.0
```

El tag es **anotado** (no *lightweight*): lleva autor, fecha y mensaje propios, que es lo que corresponde para marcar una entrega. Apunta a `d1baff1`, el commit de `main` resultante del merge del PR #3 — o sea, el estado *con el conflicto ya resuelto*.

4. Evidencias

Las cuatro capturas marcadas con 📸 en la guía, tomadas en el instante en que ocurrió cada hecho. Tres de ellas son irrepetibles: el aviso de conflicto desaparece del PR al resolverlo, y los marcadores desaparecen del archivo al borrarlos —que es, literalmente, lo que significa resolver el conflicto.



```
arielschwindt@macariel ~/SourceCode/tp01live [?] ? main [?] echo "test" >> README.md
arielschwindt@macariel ~/SourceCode/tp01live [?] ? main [?] git commit -am "test: intento de p
ush directo"
[main 142e15b] test: intento de push directo
1 file changed, 1 insertion(+)
arielschwindt@macariel ~/SourceCode/tp01live [?] r main [?] git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 328 bytes | 328.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote:
remote: - Changes must be made through a pull request.
To https://github.com/arielsch74/tp01live.git
! [remote rejected] main -> main (protected branch hook declined)
error: failed to push some refs to 'https://github.com/arielsch74/tp01live.git'
x arielschwindt@macariel ~/SourceCode/tp01live [?] r main [?]
```

Evidencia 1 — Push directo a `main` rechazado por el servidor.

Ventana de terminal real en el instante del rechazo. El `commit` local se hizo; lo que falló es el `push`. La política vive en GitHub, no en la máquina, y con `enforce_admins` activo alcanza también al dueño del repositorio.

arielsch74 / tp01live

Code Issues Pull requests 1 Agents Actions Projects Wiki Security and quality Insights Settings

Título del README: "Proyecto IngSoft3 - versión B" #3

arielsch74 wants to merge 1 commit into main from feature/titulo-b

Conversation 0 Commits 1 Checks 0 Files changed 1 +1 -1

arielsch74 commented 1 minute ago Owner

Qué cambia

Reemplaza el título del README.md (primera línea) por # Proyecto IngSoft3 - versión B.

Por qué

Misma motivación que el PR #2: el título por defecto no dice nada del proyecto.

Nota

Rama hermana de feature/titulo-a: las dos salieron del commit 301f094 de main y tocan la misma línea. Mientras ninguna esté mergeada, GitHub muestra las dos como mergeables — el conflicto no nace al escribir el cambio, nace al intentar integrarlo.

docs: renombra el título del README a Proyecto IngSoft3 - versión B 41a496a

This branch has conflicts that must be resolved

Use the [web editor](#) or the command line to resolve conflicts before continuing.

README.md

[Resolve conflicts](#)

Squash and merge You can also merge this with the command line. [View command line instructions.](#)

Reviewers: No reviews. Still in progress? [Convert to draft](#)

Assignees: No one—[assign yourself](#)

Labels: None yet

Projects: None yet

Milestone: No milestone

Development: Successfully merging this pull request may close these issues. None yet

Notifications: [Unsubscribe](#) Customize

You're receiving notifications because you authored the thread.

Evidencia 2 — El PR #3 no se puede mergear: conflicto.

Chapa *Merge conflicts* y el cartel «This branch has conflicts that must be resolved», con README.md como archivo en conflicto y el botón de merge deshabilitado. Estado irrepitable: desaparece al resolver.

README.md 1 conflict Prev Next Mark as resolved

Accept current change | Accept incoming change | Accept both changes

```

1 <<<<<< feature/titulo-b (Current change)
2 # Proyecto IngSoft3 - versión B
3 =====
4 # Proyecto IngSoft3 - versión A
5 >>>>>> main (Incoming change)
6
7 ## Instalación
8
9 ```bash
10 git clone https://github.com/arielsch74/tp01live.git
11 cd tp01live
12 ```
13

```

Keyboard hints

Evidencia 3 — Los marcadores del conflicto.

Editor de conflictos de GitHub antes de tocar nada. Arriba la versión de la rama actual, abajo la que ya estaba en main. El resto del archivo aparece sin marcar: Git fusionó solo todo lo que no chocaba.

v1.0.0 LatestCompare   arielsch74 released this now  v1.0.0  d1baff1 

Primera versión estable del repositorio. No entrega funcionalidad de producto: entrega el flujo de trabajo con el que va a entrar toda la funcionalidad del resto del semestre.

Qué incluye

- `.gitignore` base en la raíz — dependencias, artefactos de build y secretos quedan fuera del control de versiones desde el commit 2.
- `main` protegida sin bypass — `required_pull_request_reviews` con 0 aprobaciones obligatorias y `enforce_admins: true`. La regla alcanza también al dueño del repositorio: el push directo está verificado como rechazado (`GH006 / protected branch hook declined`).
- Tres Pull Requests mergeados con squash, cada uno con su descripción de qué cambia y por qué:
 - [#1](#) — sección de instalación en el README (incluye una corrección de formato detectada leyendo el propio diff).
 - [#2](#) — título del README, versión A.
 - [#3](#) — título del README, versión B, que requirió resolver un conflicto de merge a mano.
- Un conflicto real, provocado y resuelto — dos ramas nacidas del mismo commit reescribieron la misma línea. Se resolvió decidiendo el contenido (ganó la versión B) y borrando los marcadores, no con un "aceptar la mía".

Por qué 1.0.0 y no 0.1.0

SemVer dice que `1.0.0` marca el momento en que existe algo estable de lo que otros pueden depender. Acá lo estable es el contrato de trabajo: **todo entra por Pull Request, nada se pushea directo a `main`**. Ese contrato ya está configurado y verificado, así que es 1.0.0.

Los próximos cambios que sumen funcionalidad compatible serán `1.1.0`; las correcciones, `1.0.1`. Un cambio de las reglas de integración que rompa el flujo de quien ya trabaja sobre el repo sería `2.0.0`.

▼ Assets 2

 Source code (zip)	now
 Source code (tar.gz)	now

Evidencia 4 — Release `v1.0.0` publicada.

Release sobre el tag `v1.0.0`, apuntando al commit de `main` con el conflicto ya resuelto, con notas que explican qué incluye la versión y por qué el número es `1.0.0`.

5. Preparación para la defensa oral

El enunciado adelanta las preguntas y advierte que el TP se aprueba **sólo si se puede explicar qué se hizo, por qué pasa lo que pasa y cómo se resolvió**. Abajo, cada pregunta respondida contra los datos concretos de este repositorio, con lo que conviene tener abierto en pantalla al responderla.

1. ¿Para qué sirve proteger `main` si en el equipo «se tienen confianza»? ¿Qué pasó cuando intentaste pushear directo?

La confianza no es el problema: el problema es el error involuntario, y que un acuerdo que sólo vive en la cabeza de la gente no se puede auditar ni sobrevive a una incorporación nueva. La protección convierte el acuerdo del equipo en una regla que la plataforma hace cumplir — *policy as configuration*, el mismo patrón que un pipeline aplica a los tests.

Cuando intenté pushear directo, el `commit` local funcionó y el `push` fue rechazado con `GH006: Protected branch update failed` y `! [remote rejected] main -> main (protected branch hook declined)`. Dos cosas para señalar: el rechazo ocurre **en el servidor** —mi `commit` local quedó hecho y hubo que descartarlo con `git reset --hard HEAD~1` —, y me rechazó **a mí**, que soy el dueño del repositorio, porque la regla está con `enforce_admins`. Una protección que el admin puede saltar es de adorno.

Qué mostrar en pantalla: Evidencia 1 · Settings → Branches · `gh api repos/.../branches/main/protection`

2. ¿Qué es una rama *realmente* para Git? ¿Y por qué el merge que hiciste en GitHub no aparecía en tu máquina hasta que hiciste `pull`?

Una rama es **un puntero móvil a un commit**: un archivo de 41 bytes con un hash adentro. No es una copia de los archivos ni una carpeta. Por eso crear una rama es instantáneo y gratis, y por eso el flujo moderno crea ramas sin miedo. Al commitear, el puntero de la rama actual avanza; `HEAD` dice en cuál estoy parado.

El merge no apareció solo porque **mi clon y GitHub son dos repositorios completos e independientes**.

Cuando aprieto «Merge» en la web, el que avanza es el puntero `main` del servidor. Mi `main` local sigue apuntando al commit viejo hasta que traigo las novedades: `fetch` las baja sin tocar mi trabajo, `pull` es `fetch + merge`. Olvidarse este paso es la causa número uno del próximo conflicto tonto — y es, de hecho, exactamente lo que se hace a propósito en el ejercicio del conflicto.

Qué mostrar en pantalla: `git log --oneline --graph --all` antes y después del `pull`

3. ¿Qué pasa cuando dos personas modifican la misma línea? Mostrame tu conflicto y explicame por qué Git no pudo resolverlo solo.

Git fusiona con un **merge de tres vías**: compara las dos puntas contra el **ancestro común**. Si un solo lado cambió una región, la aplica sin preguntar; si la cambiaron los dos y distinto, se detiene. No es una limitación del algoritmo: la pregunta que queda —¿cuál de los dos títulos es el correcto?— no tiene respuesta técnica. Es contenido, y el contenido lo decide una persona.

En mi repositorio: `feature/titulo-a` y `feature/titulo-b` salieron las dos del commit `301f094` —el `compare` de la API lo confirma— y cambiaron la misma primera línea del `README.md`. Mergeé A, y B quedó en conflicto. Lo resolví **a mano**: dejé la versión B, borré las tres líneas de marcadores y verifiqué con `grep` que no quedara ninguno antes de `commit`ear (`1edc426`). No usé «aceptar la mía» porque eso no resuelve el conflicto: decide de antemano que el otro lado no importa.

Detalle que vale la pena mostrar: el resto del `README` —la sección `Instalación`— **no** tiene marcadores. El conflicto no fue el archivo: fue esa línea.

Qué mostrar en pantalla: Evidencias 2 y 3 · PR #3, pestaña *Commits* · `gh api repos/.../compare/<p1>...<p2>`

4. El code review es la práctica central de un equipo y en este TP trabajaste solo: ¿qué buscarías vos en el Pull Request de un compañero, y qué NO discutirías nunca?

Buscaría: que el cambio haga lo que dice el título (que el diff no traiga cosas de contrabando); que se entienda sin que el autor esté al lado, porque el que lo va a mantener puede ser otro; que tenga tests si toca comportamiento; que no introduzca riesgos de seguridad o performance; y que sea coherente con el diseño que ya existe en vez de inventar una forma nueva de hacer lo mismo.

No discutiría nunca: estilo y formato — comillas, indentación, dónde va la llave. Eso se automatiza con un linter y un formateador. Discutirlo entre humanos gasta el capital del review en lo único que una máquina hace mejor, y encima genera roce.

El review además no es sólo cazar bugs: reparte conocimiento y convierte el código en propiedad del equipo. Y la regla práctica que más importa: **PRs chicos**. Uno de 100 líneas recibe un review de verdad; uno de 3.000 recibe un «LGTM» sin leer.

Un ejemplo concreto de este repositorio: revisando el diff del PR #1 encontré que el `README` original no terminaba en salto de línea, así que el encabezado nuevo había quedado pegado al título. Nada lo marcaba como roto; sólo se ve leyendo el diff. Se corrigió con un segundo commit en la misma rama, antes de `merge`ear.

Qué mostrar en pantalla: Los PRs del repositorio con sus descripciones · el segundo commit del PR #1

5. ¿Qué significa el número de versión que le pusiste al tag?

`v1.0.0` es **SemVer**: `MAJOR.MINOR.PATCH`. `MAJOR` sube cuando rompo compatibilidad («si actualizás, algo tuyo puede dejar de andar»); `MINOR` cuando agrego funcionalidad que no rompe nada; `PATCH` cuando corrijo un bug sin cambiar el comportamiento esperado.

Lo estable que este `1.0.0` marca no es una funcionalidad: es **el contrato de trabajo** —todo entra por Pull Request, nada se pusha directo a `main`—, que a esa altura ya estaba configurado y verificado.

El número no es decorativo: es **información para el que consume** el software. Y la diferencia entre tag y release: el **tag** es una etiqueta inmutable sobre un commit («acá estaba el código de la `v1.0.0`»); la **release** le agrega la comunicación — qué cambió, escrito para humanos. Mi tag es *anotado* (lleva autor, fecha y mensaje propios) y apunta al commit `d1baff1`, el `main` con el conflicto ya resuelto.

Qué mostrar en pantalla: La página de la release · `git show v1.0.0` · `gh release view v1.0.0`

Recorrido sugerido para la defensa en vivo

- **Empezar por Settings → Branches**: mostrar la regla, y en particular las cero aprobaciones y el *Do not allow bypassing*. Explicar por qué cada tilde está donde está antes de mostrar ninguna consecuencia.
- **Después la evidencia 1**: el rechazo. Es la consecuencia de lo anterior, y lo demuestra sobre uno mismo.
- **Recorrer el PR #3 entero**: la descripción, los commits (ahí está el de merge con dos padres) y el diff. Es el lugar donde vive la historia del conflicto, porque el squash la aplanar en `main`.
- **Cerrar en la release**: qué significa `1.0.0` y por qué un tag sin notas no es lo mismo que una release.
- **Si preguntan por la IA**: `decisiones.md` §3 tiene la declaración literal —qué se delegó, qué no— y la tabla de cómo se verificó cada resultado contra el repositorio.

Punto débil conocido, dicho de frente. El tag `v1.0.0` se creó siguiendo el orden de la guía (§4.7 antes de §4.8), así que apunta a un commit que todavía no incluye `evidencias.md` ni `decisiones.md`. Es deliberado y está explicado en las notas de la release: `v1.0.0` marca el punto donde quedó operativo el **flujo de trabajo**, y los dos archivos son documentación *sobre* ese flujo. Si se considerara que forman parte del artefacto entregado, lo correcto sería un `v1.1.0` — funcionalidad nueva compatible— y no mover el tag existente, porque un tag es inmutable por definición.